

Техническое задание

1. Задание

Разработать клиент-серверное приложение для мониторинга, агрегации и анализа журналов событий (логов). Серверная часть отслеживает изменения в указанных директориях, выполняет потоковый парсинг файлов, вычисляет статистические метрики и математические модели, а также транслирует данные подключённым клиентам. Клиентская часть устанавливает TCP-подключение к серверу, отображает события в реальном времени, предоставляет инструменты фильтрации, поиска, визуализации трендов и экспорта отчётов.

Проект рассчитан на совместную работу двух человек: один развивает серверную логику (чтение файлов, математика, сетевой сервер), второй — клиентскую часть (Qt-интерфейс, сетевой клиент, визуализация, экспорт). Взаимодействие сторон синхронизируется через единый протокол обмена данными.

2. Требования

Серверная часть (Backend)

Мониторинг и чтение файлов:

- Отслеживание директорий на появление/изменение файлов форматов .log, .txt, .csv.
- Потоковое чтение новых строк (аналог tail -f) без полной загрузки файла в память.

Парсинг и коллекции:

- Извлечение структурных полей: метка времени, уровень события, код, сообщение.
- Использование регулярных выражений или фиксированных масок для разбора.
- Хранение временных данных в `std::deque` (кольцевой буфер последних N записей) и `std::map<std::string, std::vector<int>>` для агрегации по уровням/кодам.

Математическая обработка:

- Расчёт экспоненциально сглаженного среднего (EMA) частоты событий за скользящее окно.
- Вычисление перцентилей (P90, P95, P99) для интервалов между событиями.
- Аппроксимация тренда количества ошибок методом наименьших квадратов (линейная регрессия) с краткосрочным прогнозом.
- Алгоритмы реализуются самостоятельно без сторонних математических библиотек.

Сетевое взаимодействие:

- TCP-сервер (QTcpServer), асинхронная обработка множественных подключений (QTcpSocket).
- Поддержка команд протокола: SUBSCRIBE (подписка на директорию), QUERY (запрос исторических данных), UNSUBSCRIBE, PING.
- Автоматическая отправка агрегированных метрик и пуш-уведомлений при превышении настроенных порогов.

Клиентская часть (Frontend)

Подключение и управление подписками:

- Ввод адреса сервера, установка/разрыв соединения.
- Выбор директорий на сервере для мониторинга, настройка интервала обновления.

Отображение и анализ:

- Табличное представление последних событий с сортировкой по времени/уровню.
- Визуализация графиков (EMA, тренд ошибок) и сводной статистики.
- Фильтрация по диапазону дат, уровню лога, коду события, ключевым словам.

Сетевое взаимодействие:

- TCP-клиент (QTcpSocket), парсинг входящих JSON/бинарных пакетов.
- Обработка пуш-уведомлений, переподключение при обрыве связи, кеширование последних данных для стабильности UI.

3. Интерфейс (UI)

Требования применимы к клиентской части; серверная часть может использовать консольный вывод или минимальное Qt-окно статуса.

Главное окно клиента должно включать:

- **Панель подключения:** поля IP, Port, кнопки «Подключиться»/«Отключиться».
- **Панель подписки:** выбор пути директории на сервере, кнопка «Подписаться».
- **Поле ввода поискового запроса** (QLineEdit) и панель фильтров (выпадающие списки, выбор дат).
- **Кнопки быстрого доступа:** «Обновить», «Очистить таблицу», «Экспорт».
- **Область вывода результатов:** QTableView (события) + графический виджет (графики метрик).
- **Панель меню (MenuBar):**
 - Файл → Выход, Экспорт в CSV;
 - Настройки → Параметры подключения, Пороги уведомлений;
 - Справка → О программе.
- **Контекстное меню (по правому клику в таблице):**
 - «Копировать строку»;
 - «Открыть исходный файл на сервере»;
 - «Добавить код/уровень в фильтр».
- **Горячие клавиши:** Ctrl+F — фокус на поиск, F5 — принудительное обновление/переподключение, Ctrl+E — экспорт, Ctrl+C — копировать выделенное.

Интерфейс должен корректно масштабироваться при изменении размеров окна (использование QSplitter, QHBoxLayout/QVBoxLayout, растягиваемые виджеты).

4. Структура проекта

Код должен содержать подробные комментарии к каждой значимой функции, поясняющие назначение, входные/выходные данные и алгоритмические шаги.

Проект разделяется на независимые модули. Git-репозиторий должен отражать совместную работу: использование веток, мерж-запросы (или ручные мержи), разрешение конфликтов. История коммитов — не менее 5 осмысленных этапов.

Обязательные блок-схемы (подробность до основных пошаговых узлов):

- Мониторинг директорий и потоковый парсинг логов на сервере.
- Расчёт статистических метрик и обновление агрегатов (EMA, перцентили,

- регрессия).
- Сетевое взаимодействие (цикл команды сервера ↔ обработка на клиенте).
- Фильтрация и отображение данных в клиентском интерфейсе.

Структура каталогов:

```
LogWatch/
├── bin/                <-- скомпилированные .exe + необходимые .dll
├── doc/                <-- блок-схемы алгоритмов (PNG/SVG/PDF)
├── src/
│   ├── server/
│   │   ├── main.cpp
│   │   ├── filewatcher.h/.cpp    <-- мониторинг и чтение файлов
│   │   ├── logparser.h/.cpp      <-- парсинг строк, regex
│   │   ├── statsengine.h/.cpp    <-- математика (EMA, P90/95/99,
регрессия)
│   │   ├── tcpserver.h/.cpp      <-- сетевой модуль сервера
│   │   └── utils.h/.cpp          <-- вспомогательные функции
│   └── client/
│       ├── main.cpp
│       ├── mainwindow.h/.cpp     <-- главное окно UI
│       ├── networkclient.h/.cpp  <-- сетевой модуль клиента
│       ├── datamodel.h/.cpp      <-- модель для QTableView и графиков
│       └── exportutils.h/.cpp     <-- экспорт в CSV
├── .git/               <-- репозиторий Git
├── .gitignore
└── LogWatch.sln        <-- решение Visual Studio 2022
```

5. Условия разработки

- Приложение разрабатывается в среде **Visual Studio 2022**.
- Язык программирования: **C++** (стандарт **C++14** или **C++17**), фреймворк **Qt версии 5**.
- Целевая ОС: семейство **Windows 7 и выше (x64)**.
- Использование системы контроля версий Git обязательно.** Во время работы требуется вести общую историю проекта с разделением на ветки `server-dev` и `client-dev`, регулярными мержами в `master`.

6. Дополнительно (по желанию)

- Поддержка регулярных выражений в клиентском поиске и серверном парсере.
- Визуализация графиков в реальном времени без полной перерисовки окна (инкрементальное обновление).
- Логирование действий сервера и клиента в отдельные текстовые файлы с ротацией.
- Простая авторизация подключения (токен/пароль) и базовое шифрование трафика.
- Кеширование последних 1000 событий на клиенте для просмотра при временной потере связи с сервером.

7. Предъявляемые материалы

По окончании практики необходимо предоставить:

- **Директорию bin/** с готовыми исполняемыми файлами клиента и сервера, а также всеми необходимыми динамическими библиотеками (.dll) и конфигурационными файлами, достаточными для запуска на «чистой» Windows-машине.
- **Директорию src/** с исходным кодом проекта, строго соответствующую указанной структуре, содержащую подробные комментарии к классам, методам и нетривиальным участкам логики.
- **Директорию doc/** с блок-схемами алгоритмов работы приложения, детализированными до основных пошаговых узлов (чтение → парсинг → математика → сеть → UI). Форматы: PNG, SVG или PDF.
- **Ссылку на Git-репозиторий** или архив .git, демонстрирующий историю разработки (не менее 5 содержательных коммитов, отражающих этапы: инициализация, реализация парсинга/математики, реализация сети, реализация UI, финальная отладка и сборка).